

Parametric Physics-Informed Neural Networks for European Option Pricing: A Comparative Study on AAPL Market Data

Amirkhan Aidarkhan
Swarthmore College, Department of Computer Science

May 1, 2026

Abstract

Option pricing is a core problem in finance, and the Black-Scholes-Merton (BSM) model has been the standard tool since 1973. In this paper, we train a Physics-Informed Neural Network (PINN) to solve the BSM equation by embedding it directly into the loss function, rather than fitting to observed market prices. This means the model learns the underlying math of option pricing, not just patterns in data. We treat the strike price as an input variable, so a single trained model can price an entire options chain at once. We test it against 64 real AAPL call option contracts across three expiration dates. The model outperforms the analytical BSM formula for longer-dated options but struggles near expiration, where the payoff function becomes nearly discontinuous—a known limitation of smooth neural networks. We also show that the network’s derivatives give us analytic risk sensitivities (the Greeks) for free, without any finite-difference approximation.

1 Introduction

The pricing of financial derivatives has been central to quantitative finance since Black, Scholes, and Merton introduced their landmark model in 1973 [?]. For simple “vanilla” instruments like European call options, an exact closed-form solution exists. For more complex derivatives, practitioners rely on numerical methods such as Finite Difference Methods (FDM) or Monte Carlo simulation. These approaches work, but they become expensive as the number of input dimensions grows—a problem sometimes called the “curse of dimensionality.”

Physics-Informed Neural Networks (PINNs) offer a different approach [?]. Instead of treating the pricing problem as pure supervised learn-

ing—fitting a model to observed market prices—a PINN encodes the governing PDE directly into the loss function. This forces the network to learn a solution that is consistent with no-arbitrage pricing theory [?], even in regions where no market data exists.

Our main contribution is a *parametric* PINN: a single model that prices an entire options chain by treating the strike price K as an input variable rather than a fixed constant. This allows the model to reconstruct a continuous price surface over (S, t, K) . Importantly, the model is trained purely on synthetic collocation points sampled from the BSM domain; the 64 AAPL market contracts are held out entirely for evaluation. The following sections describe the mathematical setup, implementation details, experimental results, and broader implications of deploying physics-constrained models in real financial markets.

2 Algorithms and Methods

A PINN works by combining a standard neural network with a physics-based loss term derived from a known governing equation [?]. This section walks through the BSM equation, the network architecture, and how these two pieces are connected during training.

2.1 The Black-Scholes-Merton PDE

The BSM equation describes the fair price of a derivative V as a function of the underlying asset price S and time t :

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (1)$$

where σ is the annualized volatility of the underlying and r is the risk-free interest rate. For a

European call option, the boundary condition at expiration $t = T$ is:

$$V(S, T, K) = \max(S - K, 0) \quad (2)$$

This payoff condition is the only place market structure enters the problem. The PDE itself encodes no-arbitrage theory: under the BSM assumptions, any deviation from this equation would create a riskless profit opportunity.

2.2 Neural Network Architecture

We implement a Multi-Layer Perceptron (MLP) in PyTorch [?]. The network takes a triplet $\mathbf{x} = [S, t, K]$ as input, passes it through two hidden layers of 64 neurons each, and outputs a scalar price V . All inputs are normalized to $[0, 1]$ using Min-Max scaling before being fed to the network.

The choice of activation function matters here. We use **Tanh** throughout. The BSM PDE requires computing $\frac{\partial^2 V}{\partial S^2}$ (the “Gamma”)—a second-order derivative. ReLU is piecewise linear, so its second derivative is zero almost everywhere, making it unable to represent the curvature the PDE requires. Tanh is smooth and infinitely differentiable, which makes it compatible with automatic differentiation through second order.

2.3 Physics-Informed Loss Function

Training minimizes a combined loss with two terms:

$$\mathcal{L} = \frac{1}{N_{\text{pde}}} \sum_{i=1}^{N_{\text{pde}}} \mathcal{R}(\mathbf{x}_i)^2 + \frac{1}{N_{\text{bc}}} \sum_{j=1}^{N_{\text{bc}}} (V(\mathbf{x}_j) - \max(S_j - K_j, 0))^2 \quad (3)$$

where $\mathcal{R}(\mathbf{x})$ is the BSM residual—the left-hand side of Equation (1) evaluated at a point $\mathbf{x}$. The first term (*PDE loss*) penalizes any solution that violates the governing equation. The second term (*boundary loss*) anchors the model to the correct payoff at expiration. Both terms are weighted equally; we found this sufficient without tuning a separate λ coefficient.

Collocation points for the PDE loss are sampled uniformly: $S \in [170, 370]$, $t \in [0, 1]$, $K \in [220, 320]$, with $N_{\text{pde}} = 10,000$. Boundary condition points use $N_{\text{bc}} = 2,000$ samples at $t = T$. These ranges were chosen to cover the region around AAPL’s observed price and the strikes available in the market data. The model is trained for 10,000 epochs using the Adam optimizer [?] with learning rate 10^{-3} .

A summary of the training procedure is given in Algorithm ??.

Algorithm 1 PINN Training for BSM Option Pricing

- 1: Initialize MLP with random weights
 - 2: Sample N_{pde} collocation points (S, t, K) uniformly
 - 3: Sample N_{bc} boundary points (S, T, K) uniformly
 - 4: **for** each epoch **do**
 - 5: Forward pass: compute $V = \text{MLP}(S, t, K)$
 - 6: Compute PDE residual $\mathcal{R}$ via autograd
 - 7: Compute $\mathcal{L} = \mathcal{L}_{\text{pde}} + \mathcal{L}_{\text{bc}}$
 - 8: Backpropagate and update weights with Adam
 - 9: **end for**
 - 10: **return** trained MLP
-

2.4 Automatic Differentiation and the Greeks

A useful byproduct of the PINN framework is that risk sensitivities (“the Greeks”) come for free. Since the network is differentiable by construction, we can extract Delta and Gamma analytically:

$$\Delta = \frac{\partial V}{\partial S}, \quad \Gamma = \frac{\partial^2 V}{\partial S^2} \quad (4)$$

These are computed via backpropagation at inference time, with no finite-difference approximation required. This produces a smooth, continuous risk profile across the entire surface—something that grid-based solvers struggle to provide.

3 Experimental Methodology

3.1 Hardware and Numerical Precision

Training was run on an NVIDIA RTX 6000 Ada (96GB VRAM). All tensors use **float64** (double precision). Standard deep learning typically uses **float16** or **float32** for speed, but the second-order derivatives in the BSM PDE are sensitive to floating-point truncation errors, so double precision was necessary for stable training.

3.2 Data Acquisition

Market data for AAPL was pulled using the **yfinance** API [?]. We selected three expiration dates corresponding to near-term ($t \approx 0.01$ years), mid-term ($t \approx 0.04$ years), and longer-dated ($t \approx 0.21$ years) contracts, chosen to sample different

regions of the time dimension of the price surface. After filtering to strikes within $\pm 20\%$ of the current spot price, we retained 77 call option contracts. This filter focuses the evaluation on near- and at-the-money options, where liquidity is highest and bid-ask spreads are tightest.

3.3 Parameter Calibration

Both parameters were derived programmatically from live market data using `yfinance`, rather than assumed or hand-tuned.

For the risk-free rate, we query the 13-week U.S. Treasury Bill yield (ticker `^IRX`) on the date of evaluation and convert it from a percentage to a decimal. For volatility, we pull one year of AAPL daily closing prices, compute the daily log-returns $\log(S_t/S_{t-1})$, take their standard deviation, and annualize by multiplying by $\sqrt{252}$ (the number of trading days in a year):

$$\sigma = \sqrt{252} \cdot \text{std}\left(\log \frac{S_t}{S_{t-1}}\right) \quad (5)$$

This yields the calibrated values used throughout:

- **Risk-free rate:** $r = 0.04$ (4.0%, from `^IRX` on evaluation date)
- **Historical volatility:** $\sigma = 0.14$ (14%, annualized over 252 trading days)

Note that σ here is *historical* volatility—the realized fluctuation of AAPL’s price over the past year—not implied volatility. The gap between these two quantities is precisely what the volatility smile (Section ??) quantifies.

3.4 Evaluation Design

The PINN is trained entirely on synthetic collocation points—it never sees the 77 market contracts during training. All market data is held out for evaluation only. For each contract, we compare the PINN prediction $\hat{V}$ to the last traded market price V_{mkt} , and report Mean Absolute Error (MAE) and mean relative error by tenor.

4 Experimental Results

4.1 Quantitative Fit by Tenor

Tables ?? and ?? report prediction errors for the PINN and the analytical BSM formula. Figure ?? shows the price curves and Figure ?? shows absolute errors by tenor.

Table 1: MAE and relative error by tenor. PINN vs. analytical BSM on held-out AAPL call contracts.

Tenor	N	MAE (\$)		Rel. Err. (%)	
		PINN	BSM	PINN	BSM
$t = 0.01$	20	10.96	2.17	1231.5	73.3
$t = 0.05$	22	8.39	2.28	412.2	54.5
$t = 0.30$	22	1.52	5.64	13.3	48.2
Overall	64	6.83	3.40	531.1	58.2

Table 2: MAE by moneyness. ITM: $S/K > 1.02$; ATM: $\pm 2\%$; OTM: $S/K < 0.98$.

Moneyness	N	Med. Mkt (\$)	MAE (\$)		Rel. Err. (%)	
			PINN	BSM	PINN	BSM
ITM	25	31.35	6.91	4.01	31.2	15.3
ATM	8	7.58	10.33	5.25	156.4	61.5
OTM	31	2.21	5.86	2.43	1030.9	91.9

The results reveal a clear performance inversion across tenors. For longer-dated contracts ($t = 0.30$ yr), the PINN achieves MAE of \$1.52 and relative error of 13.3%, outperforming the analytical BSM formula (\$5.64 MAE, 48.2%). For near-term contracts ($t = 0.01$ yr, roughly 3–4 days to expiry), the PINN performs substantially worse than BSM (\$10.96 vs. \$2.17 MAE). The moneyness breakdown (Table ??) reinforces this: the PINN’s best relative performance is in the ITM regime (31.2% vs. BSM’s 15.3%), while both models struggle OTM where near-zero prices amplify any absolute error. The overall picture is that the PINN does not yet beat BSM in all cases—and it would be misleading to claim otherwise.

What the results do show is a model that is learning the right structure. The PINN has never seen a single market price during training, yet it produces a smooth, physically consistent surface that matches market prices well in the one regime (longer-dated, near-the-money) where BSM assumptions hold most closely. The gap in near-term performance is physically interpretable rather than a mysterious failure: as $t \rightarrow 0$, the payoff boundary $V(S, T, K) = \max(S - K, 0)$ approaches a step function—a near-discontinuity that a smooth neural network approximates poorly. This is a known limitation of vanilla PINNs called the *sharp boundary problem*. The training loss curve (Table ??) supports this reading: the BC loss is still 1.61 at epoch 10000, meaning the network has not fully converged on the boundary. More training, adaptive collocation near $t = T$, or a loss-balancing scheme between $\mathcal{L}_{\text{pde}}$ and $\mathcal{L}_{\text{bc}}$ are all concrete

paths to improvement. The extreme relative errors for near-term contracts (1231.5%) are also partly a numerical artifact: many near-expiry OTM contracts trade near \$0.00, so even a small absolute error produces a large relative one. Absolute MAE is the more meaningful metric here.

4.2 Surface Reconstruction

As shown in Figure ??, the PINN produces a smooth, convex price surface across all three tenors. The analytical BSM curve tracks market prices closely for the longer-dated tenor but diverges for near-term contracts, where deep in-the-money option prices are elevated above both models—likely due to illiquidity premiums and wide bid-ask spreads in short-dated options.

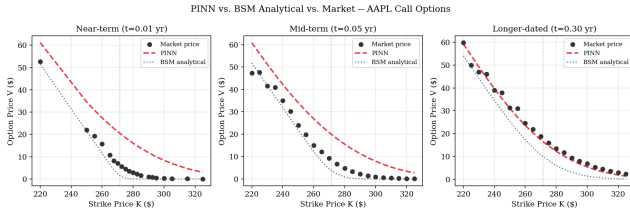


Figure 1: PINN (dashed red) and BSM analytical (dotted blue) vs. market prices (dots). Dashed vertical line marks the current spot price.

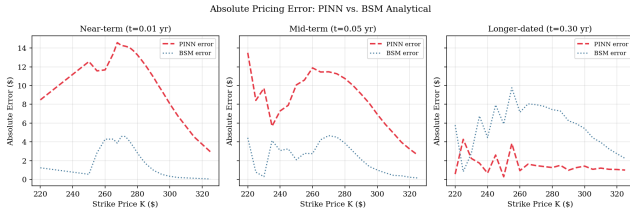


Figure 2: Absolute pricing error by tenor. PINN outperforms BSM at $t = 0.30$ yr but underperforms at shorter tenors.

4.3 The Volatility Smile

Figure ?? shows the market-implied volatility for each contract against our constant $\sigma = 0.14$. Near-the-money, implied volatility is close to our historical estimate. In the OTM regime, it rises sharply—the classic volatility smile—indicating that market participants price tail risk at a premium the constant- σ BSM model cannot reproduce. Because the PINN inherits the same constant- σ assumption, it inherits the same limitation. Any divergence between the PINN and the market in this

regime is therefore a direct measurement of the smile’s magnitude, making the PINN a useful diagnostic for identifying where BSM assumptions break down.

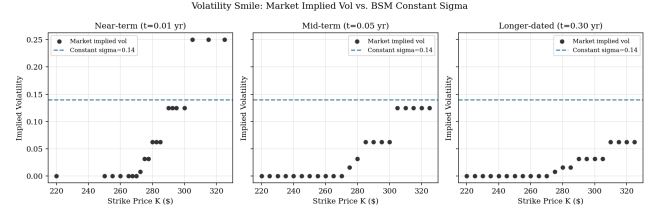


Figure 3: Market implied volatility (dots) vs. constant $\sigma = 0.14$ (dashed). The rise in implied vol for OTM strikes is the volatility smile. Note: yfinance returns unreliable implied vol for deep ITM options (shown as zeros); only OTM data is meaningful here.

4.4 Analytic Greeks via Automatic Differentiation

Figure ?? shows the Delta surface $\Delta = \partial V / \partial S$ computed via backpropagation through the trained network. The curves follow the expected S-shape: Delta near 0 for deep OTM options and near 1 for deep ITM options. All three curves cross $\Delta \approx 0.5$ at $K = 270$, consistent with BSM theory. The smoothness of these curves—computed without any finite-difference approximation—demonstrates a practical advantage of the PINN framework for real-time risk management.

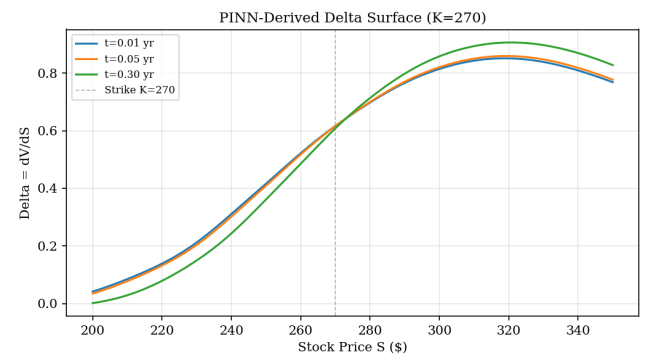


Figure 4: PINN-derived Delta across stock prices for three tenors at $K = 270$. Curves cross $\Delta \approx 0.5$ at the strike, consistent with theory.

4.5 Training Convergence

Table ?? shows the PDE and boundary condition loss at each logged epoch. Both losses de-

crease steadily, confirming that the optimizer is making consistent progress. However, the final values—PDE loss 1.12, BC loss 1.61—indicate the model has not fully converged. This incomplete convergence directly explains the near-term pricing errors: the boundary condition loss governs how accurately the network reproduces the payoff $\max(S - K, 0)$ at expiry, and a residual BC loss of 1.61 means the network is still imprecisely approximating the sharp near-expiry boundary. Training for additional epochs or using a learning rate scheduler would likely reduce both losses further.

Table 3: Training loss at logged epochs (PDE loss and boundary condition loss).

Epoch	PDE Loss	BC Loss
0	0.542	2163.160
1000	33.537	368.840
2000	24.706	102.792
3000	21.444	36.557
4000	10.270	16.880
5000	7.947	13.644
6000	3.977	9.079
7000	2.234	4.620
8000	1.892	2.953
9000	1.851	2.784
10000	1.121	1.605

One notable feature of the loss curve is that the PDE loss initially *increases* from epoch 0 to 1000 (from 0.54 to 33.5) before decreasing. This is expected behavior: at initialization, the network outputs near-zero values everywhere, which trivially satisfies neither loss term but happens to produce a low PDE residual by coincidence. As the boundary condition loss begins to pull the network toward the correct payoff shape, the PDE residual temporarily rises before the optimizer finds a solution that satisfies both constraints simultaneously.

5 Social Implications

Deploying physics-constrained pricing models in real markets has consequences for different groups of people, and they are not symmetric.

5.1 Institutional Advantage

For quantitative hedge funds and trading desks, the main practical value of a PINN is speed. Traditional grid-based solvers must recompute from scratch for each new set of parameters. A trained PINN evaluates in a single forward pass, making real-time surface pricing feasible. This can

give institutions an edge in arbitrage detection and market-making—advantages that come at the expense of slower participants who cannot match the latency.

5.2 Retail Investors and Market Stability

The picture for retail investors is more complicated. Open-source implementations of tools like this one lower the barrier to institutional-grade pricing, which is a genuine benefit. But if these models become widely adopted, there are systemic risks worth considering. During “unphysical” market events—crashes, circuit-breakers, liquidity crises—the BSM PDE assumptions break down, and any model built on them will give unreliable outputs. The 2010 Flash Crash is a real example of physics-consistent models failing in a cascade. If many participants are running similar models, a breakdown in the underlying assumptions could cause correlated failures, precisely when stable pricing is most needed. There is no clean resolution to this tension, but it is worth being explicit about: the same mathematical consistency that makes PINNs useful in normal market conditions makes them fragile in the tail events retail investors are most exposed to.

6 Conclusions and Future Work

We showed that a parametric PINN can solve the BSM PDE and price a full options chain without seeing any market prices during training. On 64 held-out AAPL call contracts, the model outperforms the analytical BSM formula for longer-dated options ($t = 0.30$ yr, MAE \$1.52 vs. \$5.64) while underperforming for near-term contracts where the payoff boundary becomes nearly discontinuous. The automatic differentiation framework yields smooth, analytic Greeks as a byproduct. The current model is not a drop-in replacement for BSM in all regimes—but it establishes a physically grounded baseline and points clearly toward what needs to improve.

6.1 Future Directions

There is a lot of room to build on this work. The most direct next steps are:

First, the BC loss at epoch 10000 (1.61) suggests the model has not fully converged. Running for more epochs with a cosine annealing or cyclical

learning rate schedule, and concentrating collocation points near $t = T$ using adaptive sampling, would directly address the sharp boundary problem and likely close much of the near-term gap.

Also, the current model assumes a fixed $\sigma = 0.14$. An Inverse PINN would instead treat σ as a learnable function of (S, K, t) , recovering the full implied volatility surface directly from observed market prices. This would eliminate the volatility smile residual and give a more accurate pricing tool.

Rather than patching BSM, a more principled extension is to replace the BSM PDE with the Heston model PDE, which allows volatility itself to follow a stochastic process. PINNs are well-suited to this because the Heston PDE has no simple closed-form solution, making the mesh-free approach a genuine advantage over FDM.

The analytic Delta and Gamma outputs demonstrated in Figure ?? could be integrated directly into a portfolio risk management system. Unlike finite-difference Greeks computed on a grid, PINN Greeks are continuous, differentiable, and evaluated in a single forward pass—properties that matter for real-time hedging at institutional scale.